DOCUMENTAÇÃO TÉCNICA

GITLOCAL Menu

Gerenciamento de repositórios GitHub por interface interativa de linha de comando.

AUTOR
Everaldo Costa Silva

VERSÃO DA DOCUMENTAÇÃO
1.0

ARQUIVO EXECUTÁVEL
gitmenu_CLIv1.sh

ANO
2026

APRESENTAÇÃO

Manual técnico

Documentação de referência para instalação, utilização, funcionalidades e considerações operacionais.

> **Objetivo:** apresentar as funcionalidades do GITLOCAL Menu, orientar sua utilização e registrar os requisitos técnicos e cuidados necessários para sua execução.

1. Identificação do programa

Propriedade	Descrição
Nome	GITLOCAL Menu
Executável	`gitmenu_CLIv1.sh`
Autor	Everaldo Costa Silva
Tipo	Aplicação interativa de linha de comando
Linguagem	Bash Shell Script
Integração	GitHub REST API e ferramentas Git
Versão documental	1.0
Licenciamento	Direitos reservados; sem licença open source declarada

2. Visão geral

O GITLOCAL Menu é uma ferramenta para gerenciamento de repositórios GitHub por meio de um menu interativo executado em terminal Linux.

O programa reúne operações relacionadas a repositórios, branches, arquivos e pastas em uma única interface. Seu objetivo é centralizar tarefas comuns de administração e movimentação de conteúdo entre o ambiente local e os repositórios remotos.

30 funcionalidades

Opções numeradas para operações de gerenciamento.

Integração GitHub

Comunicação com a API oficial do GitHub.

Interface CLI

Operação por menus e entradas interativas.

Ambiente Linux

Execução por Bash em terminal interativo.

3. Índice

1. Identificação do programa
2. Visão geral
3. Índice de funcionalidades
4. Requisitos técnicos
5. Instalação e execução
6. Autenticação e integração com GitHub
7. Configurações e arquivos temporários
8. Segurança e boas práticas
9. Limitações e considerações operacionais
10. Licenciamento e direitos autorais
11. Recomendações finais

4. Catálogo de funcionalidades

4.1 Comparação e movimentação de arquivos

01 **Comparar arquivo local e remoto**

Compara o conteúdo de um arquivo local com sua versão armazenada remotamente.

02 **Mover arquivo da MAIN**

Executa a movimentação de arquivo associado à branch principal.

03 **Mover arquivo de branch secundária**

Executa a movimentação de arquivo pertencente a uma branch secundária.

04 **Mover todos os arquivos de uma pasta**

Executa a movimentação dos arquivos associados a uma pasta selecionada.

05 **Mover todos os arquivos da MAIN**

Executa a movimentação de arquivos da branch principal.

06 **Mover todos os arquivos de branch secundária**

Executa a movimentação dos arquivos de uma branch secundária.

4.2 Consulta e pesquisa

07 **Listar repositórios e pastas**

Apresenta os repositórios e estruturas de pastas disponíveis.

08 **Pesquisar arquivo ou caminho**

Permite localizar arquivos ou caminhos dentro do contexto dos repositórios.

09 **Listar arquivos locais**

Apresenta os arquivos existentes no diretório local selecionado.

10 **Listar arquivos da MAIN**

Consulta os arquivos associados à branch principal.

11 **Listar arquivos da BRANCH**

Consulta os arquivos de uma branch secundária selecionada.

4.3 Exclusão

12 **Excluir repositório**

Disponibiliza a operação de exclusão de um repositório.

13 **Excluir branch secundária**

Disponibiliza a operação de exclusão de uma branch secundária.

14 **Excluir pasta do repositório (MAIN)**

Remove os arquivos associados à pasta selecionada na branch principal.

15 **Excluir pasta da branch secundária**

Remove os arquivos associados à pasta selecionada na branch secundária.

16 **Excluir arquivo remoto**

Disponibiliza a exclusão de um arquivo armazenado no GitHub.

4.4 Transferência de arquivos

17 **Baixar arquivo remoto**

Permite obter localmente um arquivo armazenado no repositório remoto.

18 **Enviar arquivo ao GitHub**

Envia um arquivo local para um repositório remoto.

19 **Clonar repositório de terceiros**

Disponibiliza a clonagem local de repositórios de terceiros.

20 **Enviar arquivo em anexo**

Executa o envio de arquivo selecionado como anexo.

21 **Enviar múltiplos anexos**

Executa o envio de múltiplos arquivos selecionados.

4.5 Administração de repositórios e branches

22 **Criar repositório**

Disponibiliza a criação de um novo repositório GitHub.

23 **Criar branch secundária**

Cria uma branch secundária a partir da branch principal, preparando-a para ficar sem arquivos, mantendo o histórico associado ao commit de origem.

24 **Criar pasta no repositório**

Disponibiliza a criação de uma pasta na estrutura do repositório principal.

---

25 **Criar pasta na branch secundária**

Disponibiliza a criação de uma pasta em uma branch secundária.

---

26 **Renomear repositório**

Permite alterar o nome de um repositório.

---

27 **Renomear branch secundária**

Permite alterar o nome de uma branch secundária.

---

28 **Renomear pasta da branch secundária**

Executa a operação de renomeação de uma pasta na branch secundária.

4.6 Configuração e sessão

29 **Mudar diretório local**

Altera o diretório local utilizado pelo programa. A configuração é persistida.

---

30 **Trocar usuário GitHub**

Permite encerrar a sessão atual e iniciar uma sessão com outro usuário.

> **Opção 0 — Sair:** encerra a utilização do programa e disponibiliza a saída da sessão.

5. Requisitos técnicos

5.1 Ambiente de execução

O programa foi desenvolvido para execução em ambiente Linux, utilizando Bash e ferramentas de linha de comando.

Componente	Requisito ou finalidade
Bash	Versão 4.3 ou superior recomendada, devido aos recursos utilizados pelo código.
curl	Comunicação HTTP com a API do GitHub.
jq	Processamento e interpretação de dados JSON.
base64	Codificação e decodificação de conteúdo.
git	Operações relacionadas a repositórios Git.
diff	Comparação de conteúdo entre arquivos.
cmp	Comparação binária de arquivos.
ssh-add	Recurso auxiliar relacionado à sessão SSH.
tput	Recursos de apresentação no terminal.
clear	Limpeza da tela do terminal.

> **Importante:** nem todos os componentes são obrigatórios para todas as operações. O programa utiliza ferramentas auxiliares conforme a funcionalidade executada.

5.2 Dependências

As dependências principais são `curl` e `jq`. O programa não realiza a instalação automática dos pacotes ausentes. A preparação do ambiente deve ser feita previamente pelo administrador.

5.3 Terminal interativo

O GITLOCAL Menu depende de interação com o usuário. Portanto, deve ser executado em um terminal com entrada e saída interativas. Não é indicado executá-lo como tarefa automatizada sem adaptação específica do código.

6. Instalação e execução

6.1 Preparação

Antes da execução, confirme que as dependências necessárias estão instaladas e que o ambiente possui acesso à internet.

Verifique a versão do Bash:

```
bash --version
```

Verifique as ferramentas principais:

```
command -v curl
command -v jq
command -v git
```

6.2 Permissão de execução

```
chmod +x gitmenu_CLIv1.sh
```

6.3 Inicialização

```
./gitmenu_CLIv1.sh
```

A aplicação apresentará o menu interativo. Selecione a opção desejada e forneça as informações solicitadas durante a operação.

> Execute o programa com uma conta que possua as permissões necessárias para acessar os arquivos locais e os recursos correspondentes no GitHub.

6.4 Execução global — qualquer diretório do Linux

Para executar o GITLOCAL Menu a partir de qualquer diretório do terminal, independentemente da pasta onde o arquivo esteja armazenado, crie um link simbólico em `/usr/local/bin`.

Execute os comandos abaixo, substituindo o caminho de exemplo pelo caminho real onde o arquivo está armazenado:

```
chmod +x "/caminho/ate/gitmenu_CLIv1.sh"

SCRIPT="$(realpath '/caminho/ate/gitmenu_CLIv1.sh')"
sudo ln -sfn "$SCRIPT" /usr/local/bin/gitmenu
```

Após a configuração, o programa poderá ser iniciado de qualquer diretório simplesmente digitando:

```
gitmenu
```

O link simbólico aponta diretamente para o arquivo original. Portanto, não é necessário copiar o script para `/usr/local/bin`. O arquivo permanece em sua localização original.

Se o script for movido para outra pasta, será necessário atualizar o link simbólico utilizando o novo caminho. O diretório `/usr/local/bin` normalmente já está incluído na variável `PATH` das distribuições Linux.

7. Autenticação e integração com GitHub

O programa utiliza a API REST do GitHub para realizar operações remotas. A comunicação utiliza o endereço oficial:

```
https://api.github.com
```

O código utiliza cabeçalhos de requisição compatíveis com a API:

```
Accept: application/vnd.github+json
X-GitHub-Api-Version: 2022-11-28
Authorization: Bearer [TOKEN]
```

7.1 Token de acesso pessoal

A autenticação utiliza um Personal Access Token (PAT) fornecido durante a utilização do programa.

O código trata a entrada do token sem exibi-la normalmente no terminal e mantém o valor em memória durante a sessão.

> **Proteção do token:** manter o token em memória e ocultar sua digitação não elimina todos os riscos de exposição. Processos privilegiados, comprometimento da conta local, depuração ou outras formas de acesso ao sistema podem representar riscos.

7.2 Permissões

O token precisa possuir as permissões adequadas às operações solicitadas. A permissão necessária depende do tipo de repositório, da configuração de acesso e da operação executada.

Recomenda-se utilizar um token com o menor conjunto de permissões necessário, evitando conceder privilégios desnecessários.

8. Configurações e arquivos temporários

8.1 Diretório local persistente

O diretório local selecionado pelo usuário é armazenado no caminho:

```
${XDG_CONFIG_HOME:-$HOME/.config}/github-manager/current-directory
```

Quando a variável `XDG_CONFIG_HOME` não está definida, o programa utiliza o diretório `.config` dentro da pasta pessoal do usuário.

8.2 Arquivos temporários

O programa utiliza um diretório temporário associado ao processo:

```
${TMPDIR:-/tmp}/github-manager-$$
```

O código utiliza `umask 077` e mecanismos de limpeza associados ao encerramento do processo.

> O uso de diretório temporário individualizado e permissões restritivas contribui para reduzir a exposição de arquivos temporários a outros usuários do sistema.

9. Segurança e boas práticas

9.1 Cuidados com credenciais

- Não compartilhe o PAT com terceiros.
- Não publique tokens em repositórios.
- Evite executar o programa em computadores não confiáveis.
- Revogue tokens que não sejam mais necessários.
- Utilize somente as permissões necessárias.

9.2 Operações destrutivas

Algumas opções permitem excluir repositórios, branches, pastas e arquivos. Essas operações podem causar perda de conteúdo remoto.

> Antes de executar uma exclusão, confirme o repositório, a branch, o caminho e o conteúdo que será afetado. Mantenha cópias de segurança dos dados importantes.

9.3 Operações em lote

As operações que envolvem múltiplos arquivos devem ser utilizadas com atenção. Não há garantia de rollback geral para todas as operações em lote.

Uma movimentação pode concluir a gravação no destino e falhar na remoção da origem. Nesse cenário, o conteúdo poderá existir nos dois locais e exigirá conferência e reconciliação manual.

9.4 Limite de tamanho

O código trata arquivos com tamanho superior a 100 MB como uma situação que requer outra estratégia de transferência. Para arquivos grandes, recomenda-se avaliar o uso de operações Git convencionais ou Git Large File Storage (Git LFS).

10. Limitações e considerações operacionais

10.1 Pastas em repositórios Git

O Git controla arquivos e seus conteúdos, não diretórios vazios. Para representar uma pasta sem outros arquivos, o programa utiliza o arquivo auxiliar `.gitkeep`.

10.2 Branch principal

O nome da branch principal não deve ser presumido como `main` em todos os repositórios. O código utiliza a informação de branch padrão do repositório.

10.3 Árvores Git incompletas

Quando uma consulta de árvores Git retorna indicação de resultado truncado, o programa interrompe a operação em lote para evitar trabalhar sobre uma listagem incompleta.

10.4 Operações sem rollback geral

A execução de operações remotas não representa uma transação atômica para todas as funcionalidades. Falhas de rede, permissões, limites da API ou alterações concorrentes podem deixar operações parcialmente concluídas.

10.5 Branch secundária sem arquivos

A opção 23 prepara uma branch secundária para ficar sem arquivos, mantendo seu histórico associado ao commit de origem. Isso não significa que a branch seja um repositório sem histórico.

10.6 Dependência de conectividade

As funcionalidades que consultam ou modificam repositórios remotos dependem de conexão com a internet e disponibilidade da API do GitHub.

11. Licenciamento e direitos autorais

SOFTWARE PROPRIETÁRIO

O código-fonte apresenta a indicação de direitos reservados e não declara uma licença open source.

Portanto, não se deve presumir que o programa esteja licenciado sob MIT, GPL, Apache, BSD ou qualquer outra licença de código aberto.

A ausência de uma licença open source declarada significa que não há autorização geral expressa para modificar, republicar, redistribuir ou explorar comercialmente o programa.

Qualquer autorização para essas atividades deve ser obtida expressamente do titular dos direitos.

11.1 Autoria

Autor indicado nesta documentação:
Everaldo Costa Silva.

A indicação de autoria neste documento não substitui eventual comprovação ou formalização jurídica da titularidade dos direitos.

11.2 Componentes de terceiros

O GITLOCAL Menu utiliza ferramentas e serviços de terceiros, incluindo Bash, Git, curl, jq e a plataforma GitHub. Cada componente permanece sujeito à sua própria licença, documentação e termos de utilização.

12. Recomendações finais

- Verifique as dependências antes de iniciar a aplicação.
- Utilize um token com permissões restritas.
- Confirme os destinos antes de movimentar arquivos.
- Faça backup antes de operações de exclusão.
- Confira os resultados de operações em lote.
- Não presuma que operações remotas sejam reversíveis.
- Mantenha o script em local de acesso controlado.

GITLOCAL Menu

Gerenciamento de repositórios GitHub por interface interativa de linha de comando.

Everaldo Costa Silva
Documentação técnica — Versão 1.0